

TP 02

Estructuras Repetitivas

Ciclos for · while · do-while en C

Materia	Programación I
Comisión	1.605
Fecha	28 de abril de 2026
Unidad	3 — Programación Estructurada
Profesores	Ing. Alejandro Jorge Behringer · Juan Capia

Distribución de ejercicios

#	Ciclo	Título
01	for	Contar del 1 al 10
02	for	Sumar los primeros N números
03	for	Tabla de multiplicar
04	for	Números pares del 2 al 20
05	for	Contador regresivo
06	do-while	Acumulador hasta ingresar 0
07	do-while	Validación de contraseña
08	for	Promedio de 5 notas
09	while	Números impares entre 1 y 50
10	while	Contar números positivos

⚠ **Todos los ejercicios deben cumplir los estándares de la cátedra.**

Prohibido: while(1), break en bucles, continue. Variables acumuladoras inicializadas antes del ciclo.

Ejercicio 01 · Ciclo for — Contar del 1 al 10 **Consigna**

Escribí un programa en C que muestre en pantalla los números del 1 al 10, uno por línea, usando un ciclo for.

 Salida esperada**Consola**

```
1
2
3
...
10
```

 Especificaciones técnicas

- Usar un ciclo for con variable contadora inicializada en 1.
- La condición de corte es que el contador sea menor o igual a 10.
- Imprimir el valor del contador en cada iteración con printf().
- Cada número debe aparecer en una línea separada (usar \n).

 Restricciones de cátedra

Prohibido usar while(1) o break para controlar el ciclo. Prohibido usar continue.

Ejercicio 02 · Ciclo for — Sumar los primeros N números **Consigna**

El usuario ingresa un número entero positivo N . Usando un ciclo for, calcular y mostrar la suma de todos los enteros del 1 al N .

 Ejemplo de ejecución**Entrada: $N = 5$** Ingrese N : 5

Suma del 1 al 5 = 15

Entrada: $N = 100$ Ingrese N : 100

Suma del 1 al 100 = 5050

 Especificaciones técnicas

- Leer N con `scanf()`.
- Declarar una variable acumuladora suma inicializada en 0 antes del ciclo.
- El ciclo for recorre desde 1 hasta N (inclusive) y en cada paso suma el contador al acumulador.
- Mostrar el resultado con `printf()` al terminar el ciclo.

 Restricciones de cátedra

Prohibido usar `while(1)` ni `break` en el ciclo. El acumulador debe inicializarse en 0, no en otro valor.

Ejercicio 03 · Ciclo for — Tabla de multiplicar **Consigna**

El usuario ingresa un número entero. Usando un ciclo for, mostrar la tabla de multiplicar de ese número del 1 al 10 con el formato indicado.

 Ejemplo de ejecución**Entrada: 7**

```
Ingrese un número: 7
7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
7 x 4 = 28
7 x 5 = 35
7 x 6 = 42
7 x 7 = 49
7 x 8 = 56
7 x 9 = 63
7 x 10 = 70
```

 Especificaciones técnicas

- Leer el número base con `scanf()`.
- El ciclo for recorre el multiplicador del 1 al 10.
- Calcular el producto dentro del ciclo y mostrarlo con `printf()`.
- Usar formato alineado con `%2d` para los multiplicadores y los resultados para que la tabla quede prolija.

 Restricciones de cátedra

Prohibido usar `while(1)` ni `break`. El ciclo for debe tener la condición de corte explícita en el encabezado.

Ejercicio 04 · Ciclo for — Números pares del 2 al 20 **Consigna**

Usando un ciclo for, mostrar en pantalla únicamente los números pares del 2 al 20 inclusive, en una sola línea separados por espacios.

 Salida esperada**Consola**

2 4 6 8 10 12 14 16 18 20

 Especificaciones técnicas

- Opción A: inicializar el contador en 2 y aumentarlo de a 2 en cada iteración ($i += 2$).
- Opción B: recorrer del 1 al 20 e imprimir solo cuando el número es par ($i \% 2 == 0$).
- Todos los números deben aparecer en la misma línea. Usar `printf()` sin `\n` dentro del ciclo y agregar el `\n` al final.

 Restricciones de cátedra

Si usás la Opción B, prohibido usar `continue` para saltar los impares — reemplazarlo con una estructura `if`.

Ejercicio 05 · Ciclo for — Contador regresivo **Consigna**

Escribí un programa que muestre los números del 10 al 1 en orden descendente, uno por línea, usando un ciclo for con el contador decreciente.

 Salida esperada**Consola**

```
10
9
8
...
1
```

 Especificaciones técnicas

- Inicializar el contador en 10.
- La condición de corte es que el contador sea mayor o igual a 1.
- La actualización del for debe ser decreciente: `i--` o `i -= 1`.
- Imprimir el valor en cada iteración con `printf()`.

 Restricciones de cátedra

Prohibido invertir la lógica usando una variable auxiliar que cuente de 1 a 10 y calcule `11-i`. El contador del for debe ser directamente decreciente.

Ejercicio 06 · Ciclo do-while — Acumulador hasta ingresar 0 **Consigna**

El usuario ingresa números enteros uno a uno. El programa los acumula en una suma. Cuando el usuario ingresa 0, el ciclo termina y se muestra el total acumulado. El 0 no se suma.

 Ejemplo de ejecución**Consola**

```
Ingrese un número (0 para terminar): 5
Ingrese un número (0 para terminar): 12
Ingrese un número (0 para terminar): 8
Ingrese un número (0 para terminar): 0
Total acumulado: 25
```

 Especificaciones técnicas

- Usar ciclo do-while. La condición de corte es que el número ingresado sea distinto de 0.
- Leer el número con scanf() dentro del cuerpo del do-while.
- Acumular solo si el número ingresado no es 0 (usar if dentro del ciclo).
- Declarar suma inicializada en 0 antes del ciclo.
- Mostrar el total al terminar con printf().

 Restricciones de cátedra

Prohibido usar while(1) con break como mecanismo de corte. La condición numero != 0 debe estar en el while del do-while, visible y explícita.

Ejercicio 07 · Ciclo do-while — Validación de contraseña **Consigna**

El programa solicita al usuario que ingrese una contraseña numérica. Si la contraseña no es 1234, vuelve a pedirla. El ciclo se repite hasta que el usuario ingrese el valor correcto. Al acertar, mostrar un mensaje de bienvenida.

 Ejemplo de ejecución**Consola — intento fallido y luego correcto**

```
Ingrese la contraseña: 9999
Contraseña incorrecta. Intente de nuevo.
Ingrese la contraseña: 0000
Contraseña incorrecta. Intente de nuevo.
Ingrese la contraseña: 1234
Acceso concedido. Bienvenido.
```

 Especificaciones técnicas

- Usar ciclo do-while. La condición de corte: clave != 1234.
- Leer la contraseña con scanf() dentro del cuerpo del do-while.
- Si la clave es incorrecta, imprimir el mensaje de error antes de que el ciclo vuelva a pedir el ingreso.
- Declarar la constante de la clave con #define CLAVE_CORRECTA 1234.

 Restricciones de cátedra

Prohibido usar while(1) con break. La condición clave != CLAVE_CORRECTA debe estar explícita en el while del do-while. Prohibido usar continue.

Ejercicio 08 · Ciclo for — Promedio de 5 notas **Consigna**

El programa solicita 5 notas enteras una a una usando un ciclo for. Al terminar el ingreso, calcula y muestra el promedio con dos decimales.

 Ejemplo de ejecución**Consola**

```
Ingrese nota 1: 7
Ingrese nota 2: 9
Ingrese nota 3: 6
Ingrese nota 4: 8
Ingrese nota 5: 5
Promedio: 7.00
```

 Especificaciones técnicas

- Usar un ciclo for que recorra exactamente 5 iteraciones (i de 1 a 5).
- En cada iteración, pedir la nota N con `printf()` y leerla con `scanf()`.
- Acumular la suma de las notas en una variable entera `suma`.
- Calcular el promedio fuera del ciclo. Para obtener decimales, hacer el cast: `(float)suma / 5`.
- Mostrar el resultado con `printf()` usando el especificador `%.2f`.

 Restricciones de cátedra

Prohibido usar arrays para guardar las notas — acumular directamente en la variable `suma` dentro del ciclo.

Ejercicio 09 · Ciclo while — Números impares entre 1 y 50 **Consigna**

Usando un ciclo while, mostrar todos los números impares entre 1 y 50 inclusive, en una sola línea separados por espacios.

 Salida esperada**Consola**

```
1 3 5 7 9 11 13 15 17 19 21 23 25
27 29 31 33 35 37 39 41 43 45 47 49
```

 Especificaciones técnicas

- Inicializar el contador en 1 antes del while.
- La condición del while: contador $\leq$ 50.
- Imprimir el contador solo si es impar (contador % 2 $\neq$ 0).
- Incrementar el contador en 1 al final de cada iteración.
- Agregar un salto de línea ($\backslash$ n) al finalizar el ciclo, fuera de él.

 Restricciones de cátedra

Prohibido usar continue para saltar los pares. Si el número es par, ignorarlo con un if (contador % 2 $\neq$ 0). Prohibido usar while(1).

Ejercicio 10 · Ciclo while — Contar números positivos **Consigna**

El usuario ingresa números enteros uno a uno. El programa cuenta cuántos de ellos son positivos. Cuando ingresa un número negativo, el ciclo termina y se muestra la cantidad de positivos ingresados. El número negativo no se cuenta.

 Ejemplo de ejecución**Consola**

```
Ingrese un número: 5
Ingrese un número: 0
Ingrese un número: 12
Ingrese un número: 7
Ingrese un número: -3
Cantidad de números positivos: 3
```

Número ingresado	¿Se cuenta?
Positivo (> 0)	Sí, incrementar contador
Cero ($= 0$)	No
Negativo (< 0)	No — termina el ciclo

 Especificaciones técnicas

- Leer el primer número con `scanf()` ANTES de entrar al `while` (patrón leer-antes-de-entrar).
- La condición del `while`: `numero >= 0`.
- Dentro del ciclo: si el número es estrictamente mayor a 0, incrementar el contador.
- Al final del cuerpo del ciclo, leer el siguiente número con `scanf()`.
- Mostrar el contador al terminar.

 Restricciones de cátedra

Prohibido usar `while(1)` con `break`. La condición `numero >= 0` debe estar explícita en el encabezado del `while`. Prohibido usar `continue`.